UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 5: Midterm 1 Review

Wednesday, March 4, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 2-3 to solve a few exercises.
    - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.
- Attendance will be taken at the end.

Reminders

- Midterm 1 is tonight
  - Good luck everyone! :)

Today's Goals

- Midterm Review

Exam Logistics + Advice

- Location & Time: **<u>7-9pm</u>**, see below for location (decided by *your labs start time*!)
  - **ILC S140**: **Section 2 labs of 9:05, 10:10, 11:15**
  - **ISB 135: rest of Section 2 (labs of 12:20, 1:25), all of Section 1**
- Do not prioritize doing the exam in order!
  - The order of the questions means nothing. Move on to the next question if you are stuck.
- Do not leave any question blank!
  - Don't write garbage or throw away comments. Write down notes of what you know, that might jog your memory.
- Write notes to yourself or underline important statements!
  - What does this function take in? What it returns? What is being graded?
- Ask questions!
  - If something doesn't make sense, raise your hand and a proctor will come over to help out.
- Use a pencil! If you don't have any, you can get them free at the library!

Topic Poll

- Raise your hand for your top 2-3 topics
  - Type signatures
  - Higher-order functions
  - Mental models
  - Lists
  - Testing

Type Signatures

Type Signatures

What are the types of g, a, and x?

```
const g = (a, x) => a.map(f => (x => f(f(x)))).map(f => f(x));
```

- Look at the smallest atomic parts of this function.
- Write down what you know immediately.
- Deduce the rest using a combination of what you figured out previously.

Higher-Order Functions

Higher Order Functions (HOFs)

```
function reducer(acc, e) {
    return {
        x: acc.y + e,
        y: acc.x
    };
}
const pair = a => a.reduce(reducer, {x: "", y: ""});
```

What is **`pair(["that", "is", "a", "short", "text"])`**?

Mental Models + Closures

Mental Models

For the line defining o1 below, state how many objects except { } are created when executing that line. State what values are printed. Explain your answers.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

Lists

Lists

Write a function **`sumEveryK(lst: List<number>, k: number): List<number>`** that splits **`lst`** into groups of **`k`** adjacent elements (the last group may have fewer) and returns a list with the sum of each group (in the same order). Assume **`k`** is a positive natural. Try not to create other temporary lists.

Example:

For `lst = 1,0,3,5,2,3,7` with `k = 3`

Output of `sumEveryK(lst, k)` should be `list = 4,10,7`

Testing

Testing

The `findIndex` function returns the index of the first element of array `a` for which `f` returns `true`, or `-1` if no such element exists

```
// findIndex<T>(a: T[], f: T => boolean): number
function findIndex(a, f) {
    for (let i = 0; i < a.length; ++i) {
        if (f(a[i])) { return i; }
    }
    return -1;
}
```

Write three representative cases that you would use to test `findIndex`.
You need not write code, but clearly indicate inputs, output, and the purpose of the test.

GOOD LUCK!!!

🤠